

Security Whitepaper: LLM Guardrails & Infrastructure Hardening

Protecting Production Environments and API Wallets

Preventing LLM Slop via Schema-Enforced JSON Logic Gates

When integrating Large Language Models (LLMs) into production applications, the primary risk is non-deterministic output—often referred to as "LLM slop" or hallucination. If an AI returns unstructured text when the application expects an array of ingredients, the system crashes.

To harden the Essential Eats architecture against this, the system completely bypasses conversational AI paradigms in favor of **Schema-Enforced JSON Logic Gates**.

The Mechanism of Enforcement

Instead of asking the LLM to "generate a recipe," the API acts as a strict orchestrator:

1. **Strict Prompt Engineering:** The system prompt does not converse. It provides a rigid directive outlining the exact JSON schema required. It explicitly forbids conversational filler (e.g., "Here is your recipe!").
2. **Type-Casting the LLM:** By instructing the Gemini API to output strict JSON, the LLM is constrained at the generation level to format its response as machine-readable data.
3. **Sanitization and Deduplication:** When parsing noisy data (like OCR receipt text), the prompt forces the AI into a categorization and sanitization logic gate. "Concord Grape Juice" must be mapped to the generic "Grape Juice" and assigned a predefined category (e.g., "Beverages"). This prevents inventory fragmentation.
4. **Deterministic Validation:** Upon receiving the payload, the Hono backend parses the JSON. If the payload lacks required keys (e.g., *prep_time*, *ingredients* array), the logic gate rejects the output, preventing malformed data from ever reaching the database or the presentation layer.

This approach transforms the LLM from a volatile conversationalist into a predictable, highly capable data transformation microservice.

Isolating Mutations to Firewall Production

A significant challenge in deploying portfolio applications is allowing public interaction—especially for recruiters and hiring managers—without exposing the system to malicious API abuse, database tampering, or exorbitant billing costs.

To solve this, Essential Eats utilizes a ****Decoupled Sandbox Architecture****.

The Architecture of Isolation

- **1. Browser localStorage as a Database Shard**
 - When a user accesses the sandbox, their inventory mutations (adding/removing items) are written directly to their browser's *localStorage*.
 - Instead of provisioning unique backend database rows for every public user, the entire data layer for the "Recruiter Demo" is shifted client-side.
 - This grants every user an isolated, pristine database session that persists across reloads but physically cannot intersect with the production Cloudflare D1 database.
 - Private production data is completely firewalled.
- **2. IP-Based Edge Rate Limiting**
 - While database mutations are handled client-side, requests to the AI engine (e.g., `/api/suggest`) must still hit the Gemini API via the backend. To protect the API wallet:
 - A custom rate limiter is deployed at the Cloudflare Edge, identifying users by their IP address.
 - The system enforces a strict quota (typically 5 AI generation requests per day per IP).
 - Because this logic executes on the edge before hitting the main worker process, malicious floods are dropped with near-zero latency and cost.
- **3. Aggressive Prompt Caching**
 - To further reduce API calls while maintaining the illusion of infinite generation, the system employs aggressive prompt caching at the edge. Frequently tested inventory combinations and standard recipe requests return cached JSON instantly, bypassing the LLM entirely for common sandbox interactions.

This multi-tiered defense strategy ensures the portfolio remains live, interactive, and impressive, while the underlying infrastructure and budget remain completely secure.